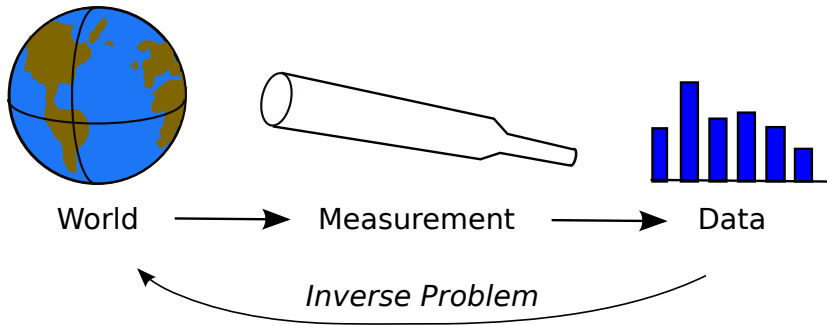
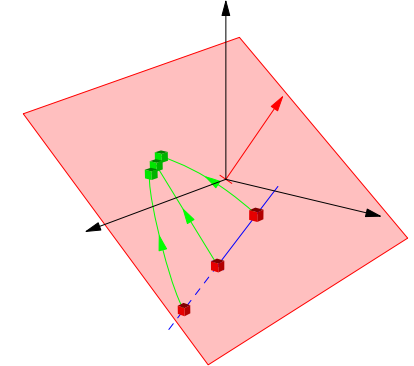


Understand Mappings



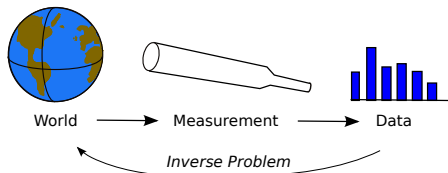
Mappings, Linear Maps, Solving Linear Systems

1. **Mappings**
2. **Linear Maps**



Transforming Data

- In the last lecture we spent time developing a sophisticated view of vector spaces and operators
- At a mathematical level machine learning can be viewed as performing an inverse mapping



- Although our mappings are not necessarily linear in either direction we learn a lot by understanding linear operators

Inverse Problems

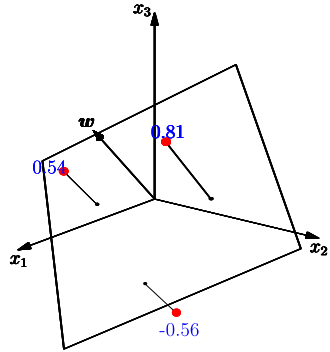
- Given m observations $\{(x_k, y_k) | k = 1, \dots, m\}$ and p unknown $w = (w_1, w_2, \dots, w_p)^T$ such that $x_k^T w = y_k$ then to find w
- Define the *design matrix* as the matrix of feature vectors

$$X = \begin{pmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_m^T \end{pmatrix} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{mp} \end{pmatrix}$$

- and the target vector $y = (y_1, y_2, \dots, y_m)^T$
- Then if $m = p$ we have $y = Xw$ or $w = X^{-1}y$

Linear Regression

- $x_k^T w$ depends on distance from separating



- If $m > p$ then X isn't square so doesn't have an inverse
- Worse unless the data is accurate $y \approx Xw \Rightarrow$ no "solution"
- Problem solved by Gauss to predict the orbit of the asteroid Ceres

Linear Least Squares

- The error of input pattern x_k is

$$\epsilon_k = x_k^T w - y_k$$

- The squared error

$$E(w|\mathcal{D}) = \sum_{k=1}^m (x_k^T w - y_k)^2 = \sum_{k=1}^m \epsilon_k^2 = \|\epsilon\|^2$$

- We can define the error vector

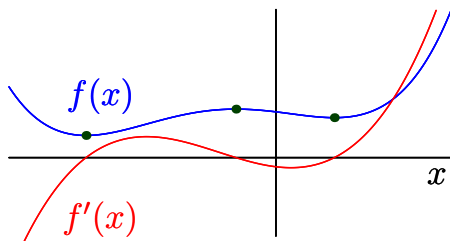
$$\epsilon = Xw - y$$

(note that $\epsilon_k = x_k^T w - y_k$)

- Minimising this error is known as the least squares problem

Finding a Minimum

- The minima of a one dimensional function, $f(x)$, are given by $f'(x) = 0$



- The minima of an n -dimensions function $f(x)$ are given by the set of equations

$$\frac{\partial f(x)}{\partial x_i} = 0 \quad \forall i = 1, \dots, n$$

Gradients

- The **grad** operator ∇ is the gradient operator in high dimensions

$$\nabla f(x) = \begin{pmatrix} \frac{\partial f(x)}{\partial x_1} \\ \frac{\partial f(x)}{\partial x_2} \\ \vdots \\ \frac{\partial f(x)}{\partial x_n} \end{pmatrix}$$

- The partial derivatives (curly d's)

$$\frac{\partial f(x)}{\partial x_i}$$

means differentiate with respect to x_i treating all other components x_j as constants

Least Squares Solution

- The least squared solution is give by

$$\begin{aligned}\nabla E(\mathbf{w}|\mathcal{D}) &= \nabla \|\epsilon\|^2 = \nabla \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 \\ &= \nabla (\mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w} - 2\mathbf{w}^\top \mathbf{X}^\top \mathbf{y} + \mathbf{y}^\top \mathbf{y}) \\ &= 2(\mathbf{X}^\top \mathbf{X} \mathbf{w} - \mathbf{X}^\top \mathbf{y}) = \mathbf{0}\end{aligned}$$

- Or

$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} = \mathbf{X}^+ \mathbf{y}$$

- $\mathbf{X}^+ = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$ is known as the pseudo inverse

Computing Gradients

- To understand gradients we sometimes need to go back to components

$$\begin{aligned}\nabla \mathbf{w}^\top \mathbf{M} \mathbf{w} &= \begin{pmatrix} \frac{\partial}{\partial w_1} \\ \frac{\partial}{\partial w_2} \\ \frac{\partial}{\partial w_3} \\ \vdots \end{pmatrix} \sum_{i,j} w_i M_{ij} w_j = \begin{pmatrix} \sum_j M_{1j} w_j + \sum_i w_i M_{i1} \\ \sum_j M_{2j} w_j + \sum_i w_i M_{i2} \\ \sum_j M_{3j} w_j + \sum_i w_i M_{i3} \\ \vdots \end{pmatrix} \\ &= \mathbf{M} \mathbf{w} + \mathbf{M}^\top \mathbf{w}\end{aligned}$$

- It is tedious to compute these things component-wise, but when you need to understand what is going on then go back to the basics

Missing Bits of the Mathematics

- Note that $\|\mathbf{a}\|^2 = \mathbf{a}^\top \mathbf{a} = \sum_i a_i^2$

$$\begin{aligned}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 &= (\mathbf{X}\mathbf{w} - \mathbf{y})^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) = (\mathbf{w}^\top \mathbf{X}^\top - \mathbf{y}^\top) (\mathbf{X}\mathbf{w} - \mathbf{y}) \\ &= \mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w} - 2\mathbf{w}^\top \mathbf{X}^\top \mathbf{y} + \mathbf{y}^\top \mathbf{y}\end{aligned}$$

- Where we have used $\mathbf{w}^\top \mathbf{X}^\top \mathbf{y} = \mathbf{y}^\top \mathbf{X} \mathbf{w}$, $\sum_{i,j} w_i X_{ji} y_j = \sum_{i,j} y_i X_{ij} w_j$

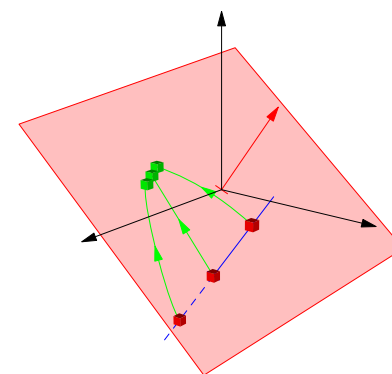
- Also $\nabla \mathbf{w}^\top \mathbf{M} \mathbf{w} = \mathbf{M} \mathbf{w} + \mathbf{M}^\top \mathbf{w}$

- If $\mathbf{M} = \mathbf{M}^\top$ (i.e. $\mathbf{M}$ is symmetric) then $\nabla \mathbf{w}^\top \mathbf{M} \mathbf{w} = 2\mathbf{M} \mathbf{w}$

- As $(\mathbf{A}\mathbf{B})^\top = \mathbf{B}^\top \mathbf{A}^\top$ then $(\mathbf{X}^\top \mathbf{X})^\top = \mathbf{X}^\top \mathbf{X}^{\top\top} = \mathbf{X}^\top \mathbf{X}$ and $\mathbf{X}^\top \mathbf{X}$ is symmetric

Outline

1. Mappings
2. Linear Maps

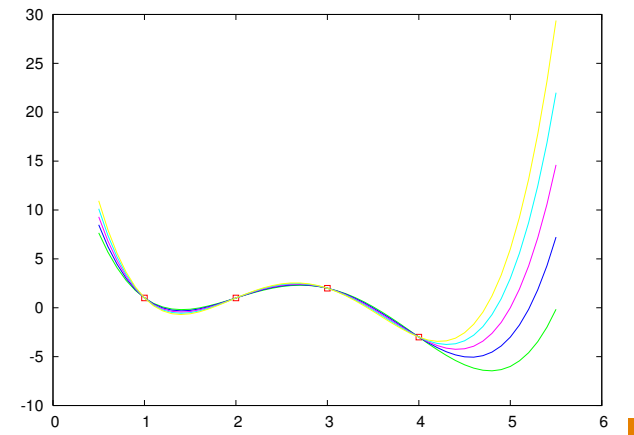


Solving Inverse Problems

- Gauss showed us how to solve **over-constrained** problems (we have more observations than parameters)
- We seek a solution which isn't necessarily exact but minimises an error
- But, what if we have more parameters than observations
- That is, we are **under-constrained**
- Note that in some directions you might be over-constrained and in other directions under-constrained
- This is very typical of most machine learning problems

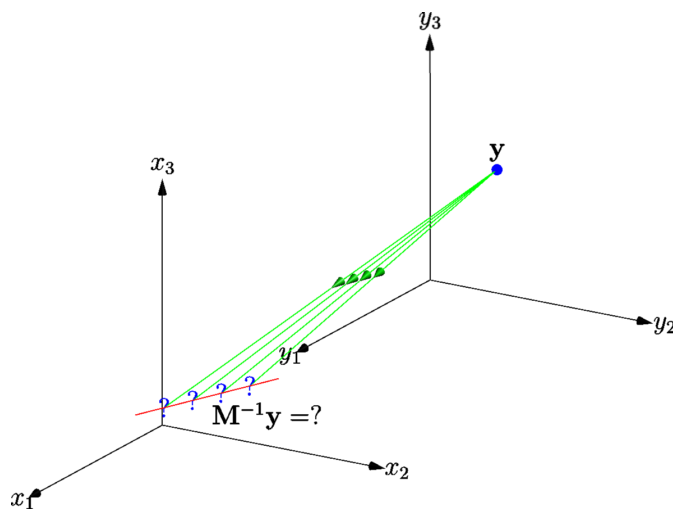
Under Constrained Systems

- If we have less data-points than parameters then there will be multiple solutions



What is the Inverse?

- Many points can map to the same points



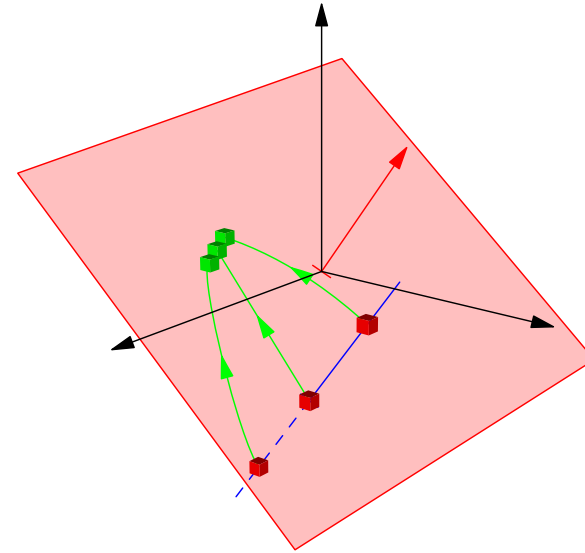
Under-constrained Systems

- The system is **under-constrained**
- We have more unknowns than equations
- The inverse is not unique
- Solving the inverse problem ($w = (X^T X)^{-1} X^T y$) is said to be **ill-posed**
- The inverse $(X^T X)^{-1}$ doesn't exist
- If we have a complicated learning machine and not sufficient data we often end with an ill-posed inverse problem (there are lots of sets of parameters that explain the data)

III-Conditions

- Singular matrices are rare (although they occur when we don't have enough data), but matrices that are close to being singular are common
- If a matrix is close to singular it is ill-conditioned
- Ill-conditioned matrices have some small eigenvalues
- All points get contracted towards a plane
- Large matrices are very often ill conditioned

III-Conditioned Matrices



III-Conditioning in ML

- Ill-conditioning in machine learning occurs when a very small change in the learning data causes a large change in the predictions of the learning machine
- In linear regression the matrix $\mathbf{X}^T\mathbf{X}$ is ill-conditioned when we have as many data points as parameters
- Much of machine learning is concerned with making learning machines better conditioned
- Adding regularisers is one approach to achieve this

Summary

- Linear mappings are commonly used in machine learning algorithms such as regression
- We will often meet the pseudo-inverse involving inverting $\mathbf{X}^T\mathbf{X}$
- They can be inherently unstable to noise in the inputs